



誠朴雄偉
勵學敦行

复习课（三）

（语义分析）

冯洋

南京大学计算机学院

fengyang@nju.edu.cn





四、简答题（共 1 题，共 15 分）

给定一个表示四进制小数的文法 G ，其开始符号为 S ，并且包含以下四个产生式。小数部分可以为空，即 123 是合法的四进制小数。产生式如下：

得分	
评分人	

$$\begin{aligned} G(S): \quad S &\rightarrow Q_1 \mid Q_1.Q_2 \\ Q_1 &\rightarrow Q_1\text{Num} \mid \text{Num} \\ Q_2 &\rightarrow \text{Num}Q_2 \mid \text{Num} \\ \text{Num} &\rightarrow 0 \mid 1 \mid 2 \mid 3 \end{aligned}$$

- 1.（5 分）文法 G 中包含左递归，请消除左递归；
- 2.（5 分）请设计一个语法制导翻译方案来统计四进制小数中奇数数字（1 和 3）的个数。
例如 2.123 中奇数数字个数为 2；
- 3.（5 分）请设计一个语法制导翻译方案将四进制小数转换为二进制字符串。例如 2.123 转换为的二进制字符串为” 10.011011”。



语法制导定义的基本概念



- 计算机程序是由很多语法结构构成
- 相同的语法单元在不同的语法结构中可能包含不同的语义信息
- 使用上下文无关文法引导语言的翻译
 - **CFG**的非终结符号代表了语言的某个构造
 - 程序设计语言的构造由更小的构造组合而成
 - 一个构造的语义可以由小构造的含义**综合**而来
 - 比如：表达式 $x+y$ 的类型由 x 、 y 的类型和运算符 $+$ 决定
 - 也可以从附近的构造**继承**而来
 - 比如：声明`int x;`中 x 的类型由它左边的类型表达式决定



语法制导定义的基本概念



■ 背景:

- 上下文无关文法只能描述语法
- 需要结合语义规则来完成语义分析与中间代码生成

■ 语法制导的定义 (**SDD: Syntax-Directed Definition**):

- 一种形式系统，将语义规则附着到文法产生式上
- 将文法符号和某些属性相关联
- 并通过**语义规则**来描述如何计算属性的值

语法树 + 属性计算 = 语义分析



语法制导定义的基本概念



■ 语法制导定义(SDD: Syntax-Directed Definition)

- 将文法符号和某些属性相关联
- 并通过**语义规则**来描述如何计算属性的值
- $E \rightarrow E_1 + T$ $E.code = E_1.code || T.code || '+'$
 - 属性code代表中缀表达式的逆波兰表示(后缀表示法)，规则说明加法表达式的逆波兰表示由两个分量的逆波兰表示并置，然后加上‘+’得到

■ 语法制导翻译(SDD: Syntax-Directed Translation)

- 在产生式体中加入**语义动作**，并在适当的时候执行这些语义动作
- $E \rightarrow E_1 + T$ {print '+';}



语法制导定义的基本概念



特性

表达方式

可读性

应用

SDD（属性定义）

抽象、数学化

高，便于教学与证明

理论建模，描述属性依赖

SDT（语义动作）

具体、嵌入代码

偏向实现，代码细节多

编译器工具（如 YACC/Bison）



语法制导定义的基本概念



- SDD是对上下文无关文法的推广
 - SDD = 上下文无关文法 + 属性 + 语义规则
 - 将每个文法符号和一个语义属性集合相关联
 - 将每个产生式和一组语义规则相关联，这些规则用于计算该产生式中各文法符号的属性值
- 为什么需要属性？
 - 语法分析只能保证程序符号串符合语法规则
 - 但程序的语义信息（如类型、值、作用域等）需要额外描述
 - 属性 (Attribute) 就是为了解决这个问题
 - 给文法符号附加额外信息
 - 在分析过程中逐步计算并传播这些信息



语法制导定义的基本概念



- 属性 (Attribute): 与文法符号相关联的值, 用来存储语义信息
- 在属性文法 (Attributed Grammar) 中, 属性之间可能存在依赖关系

$$E.val = E1.val + T.val$$



语法制导定义的基本概念



- 属性 (Attribute): 与文法符号相关联的值, 用来存储语义信息
- 在属性文法 (Attributed Grammar) 中, 属性之间可能存在依赖关系

$$E.val = E1.val + T.val$$

这里 $E.val$ 的计算依赖于 $E1.val$ 和 $T.val$

问题: 我们如何确定属性计算的顺序?

引入依赖关系的图形表示来形式化表达属性之间的依赖



语法制导定义的基本概念



- 实践中很少先构造语法分析树再进行SDD求值
- 但在分析树上求值有助于翻译方案的可视化，便于理解
- 注释语法分析树
 - 包含了各个结点的各属性值的语法分析树
- 步骤：
 - 对于任意的输入串，首先构造出相应的分析树。
 - 给各个结点（根据其文法符号）加上相应的属性值
 - 按照语义规则计算这些属性值即可



语法制导定义的基本概念



- 按照分析树中的分支对应的文法产生式，应用相应的语义规则计算属性值
- 计算顺序问题：
 - 如果某个结点 N 的属性 a 为 $f(N_1.b_1, N_2.b_2, \dots, N_k.b_k)$ ，那么我们需要先算出 $N_1.b_1, N_2.b_2, \dots, N_k.b_k$ 的值。
- 如果我们可以给各个属性值排出计算顺序，那么这个注释分析树就可以计算得到
 - S 属性的SDD一定可以按照自底向上的方式求值
- 下面的SDD不能计算
 - $A \rightarrow B \qquad A.s=B.i; \qquad B.i=A.s+1;$



语法制导定义的基本概念



- 分为两类：
 - 综合属性 (Synthesized Attribute): 自下而上计算，从子结点传到父结点，常用于表达式的值、类型计算
 - 继承属性 (Inherited Attribute): 自上而下或兄弟结点之间传递，常用于符号表传递、上下文约束、类型环境
- 不允许结点的继承属性通过子结点上的属性来定义，但是允许结点的综合属性依赖于结点本身的继承属性
- 终结符号有综合属性（由词法分析获得），但是没有继承属性



语法制导定义的基本概念



■ 综合属性示例：

○ 文法： $E \rightarrow E + T \mid T$ $T \rightarrow \text{num}$

$E \rightarrow E1 + T$ $\{ E.\text{val} = E1.\text{val} + T.\text{val} \}$

$E \rightarrow T$ $\{ E.\text{val} = T.\text{val} \}$

$T \rightarrow \text{num}$ $\{ T.\text{val} = \text{num}.\text{lexval} \}$

val 是 综合属性，计算方向是 从下到上



语法制导定义的基本概念



■ 继承属性示例：

○ 文法： $L \rightarrow T L'$ $L' \rightarrow , T L' \mid \varepsilon$ $T \rightarrow \text{id}$

$T.type$ 由父节点传递

$\text{id.entry.type} = T.type$ （把类型传给符号表项）

$T.type$ 就是 继承属性，它随文法结构向下传递



语法制导定义的基本概念



- SDD中属性的求值顺序
 - 在对SDD的求值过程中，如果结点N的属性 a 依赖于结点 M_1 的属性 a_1 ， M_2 的属性 a_2 ， $\dots$ 。那么我们必须先计算出 M_i 的属性，才能计算N的属性 a
 - 使用依赖图来表示计算顺序
 - 显然，这些值的计算顺序应该形成一个偏序关系。如果依赖图中出现了环，表示属性值无法计算



语法制导定义的基本概念



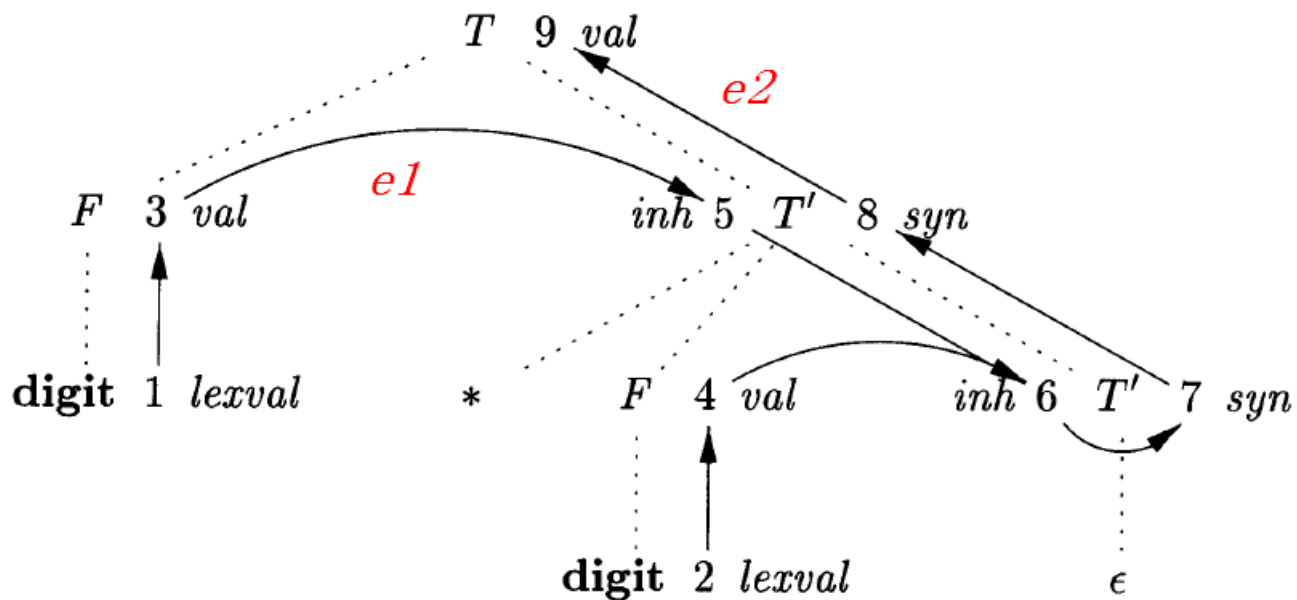
- 依赖的图形化表示：**依赖图**
- 描述了某棵特定的分析树上各个属性实例之间的信息流（计算顺序）
 - 从实例 a_1 到实例 a_2 的有向边表示计算 a_2 时需要 a_1 的值（必须先计算 a_2 ，再计算 a_1 ）
- 对于标号为X的分析树结点N，和X关联的每个属性a都对应依赖图的一个结点N. a
- 结点N对应的产生式的语义规则通过X. c计算了A. b的值，且在分析树中X和A分别对应于 N_1 和 N_2 ，那么从 $N_1. c$ 到 $N_2. b$ 有一条边



语法制导定义的基本概念



- $3*2$ 的注释分析树;
- $T \rightarrow FT' \{T.val = T'.syn; T'.inh = F.val;\}$
 - 边 $e1$ 、 $e2$
- 可能的计算顺序:
 - 1.2.3.4.5.6.7.8.9



产生式	语义规则
1) $T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * FT'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



语法制导定义的基本概念



- 各个属性的值需要按照依赖图的拓扑顺序计算
 - 如果依赖图中存在环，则属性计算无法进行
- 给定一个SDD，很难判定是否存在一棵分析树，其对应的依赖图包含环
- 怎样的SDD可以和以前所介绍的语法分析结合在一起？



语法制导定义的基本概念



- 特定类型的SDD一定不包含环，且有固定排序模式
 - S属性的SDD
 - L属性的SDD
- 对于这些类型的SDD，我们可以确定属性的计算顺序，且可以把不需要的属性（及分析树结点）抛弃以提高效率
- 这两类SDD可以很好地和我们已经研究过的语法分析相结合



语法制导定义的基本概念



- 只包含综合属性的SDD称为S属性的SDD
 - 每个语义规则都根据产生式体中的属性值来计算头部非终结符号的属性值
- S属性的SDD可以和LR语法分析器一起实现
 - 栈中的状态可以附加相应的属性值
 - 在进行归约时，按照语义规则计算归约得到的符号的属性值
- 语义规则不应该有复杂的副作用
 - 要求副作用不影响其它属性的求值
 - 没有副作用的SDD称为属性文法



语法制导定义的基本概念



- L属性的SDD，其中，每个属性
 - 要么是综合属性
 - 要么是继承属性，且产生式 $A \rightarrow X_1X_2\cdots X_n$ 中计算 X_i 的 a 的规则只能使用
 - A的继承属性
 - X_i 左边的文法符号 X_j 的继承属性或综合属性
 - X_i 自身的继承或综合属性，且这些属性之间的依赖关系不形成环



语法制导定义的基本概念



- S属性文法（只有综合属性）一定不会有环（因为信息只往上走）
- L属性文法（继承 + 综合，受限条件）也能保证不存在环路（因为继承只依赖父或左兄弟）
- 属性的继承/综合划分，帮助我们构造 S 属性文法与 L 属性文法，这两类文法天然避免了依赖图中的环
S文法与L文法是对属性依赖的限制，它们是“子集”，设计上就是为了 保证可行的无环计算顺序



语法制导定义的基本概念



- S属性的SDD: 每个属性都是综合属性
- 都是根据子构造的属性计算出父构造的属性
- 在依赖图中, 总是通过子结点的属性值来计算父结点的属性值。可以和自顶向下、自底向上的语法分析过程一起计算
 - 自底向上
 - 在构造分析树的结点的同时计算相关的属性 (此时其子结点的属性必然已经计算完毕)
 - 自顶向下
 - 递归下降分析中, 可以在过程A()的最后计算A的属性 (此时A调用的其他过程 (对应于子结构) 已经调用完毕)



语法制导定义的基本概念



- L属性的SDD：每个属性
 - 要么是综合属性
 - 要么是继承属性，且产生式 $A \rightarrow X_1X_2\cdots X_n$ 中计算 X_i 的 a 的规则只能使用
 - A 的继承属性
 - X_i 左边的文法符号 X_j 的继承属性或综合属性
 - X_i 自身的继承或综合属性，且这些属性之间的依赖关系不形成环
- 特点
 - 依赖图的边
 - 继承属性从左到右，从上到下
 - 综合属性从下到上
 - 在扫描过程中，计算一个属性值时，和它相关的依赖属性都已经计算完毕



语法制导定义的基本概念



■ 语义规则 (Semantic Rules)

- 定义：语义规则是抽象层次上的定义，用于描述属性如何从文法符号传播、计算和约束；它通常写在属性文法 (Attribute Grammar, AG) 的形式化体系中

■ 语义动作 (Semantic Actions)

- 定义：语义动作是实现层次上的指令，通常嵌入在文法产生式中的可执行代码，在语法分析过程中被触发执行

语义规则：逻辑层面的“应当如何计算属性”

语义动作：实现层面的“在编译器里怎么做”

语义规则 = 数学公式

语义动作 = 代码实现



语法制导定义的基本概念



- 副作用（Side effect）：语义规则执行时对外部运行时或全局状态的修改，例如：
 - 修改符号表（insert、update）
 - 生成/输出中间代码或目标代码（emit）
 - 写日志/报错（print/error）
 - 修改全局计数器、序号等
- 核心问题：副作用如果位置不当或在不确定的求值顺序下执行，会导致不一致或难以重现的语义错误。



语法制导定义的基本概念



- 具有受控副作用的语义规则
- 属性文法没有副作用，但增加了描述的复杂度
 - 比如语法分析时如果没有副作用，标识符表就必须作为属性传递
 - 可以把标识符表作为全局变量，然后通过副作用函数来添加新标识符



语法制导定义的基本概念



- 具有受控副作用的语义规则
- 属性文法没有副作用，但增加了描述的复杂度
 - 比如语法分析时如果没有副作用，标识符表就必须作为属性传递
 - 可以把标识符表作为全局变量，然后通过副作用函数来添加新标识符
- 受控的副作用需要满足的条件：
 - 先行条件已满足：动作执行时，它所依赖的所有属性/信息均已被正确计算。
 - 执行顺序确定：副作用的执行位置在语法/语义规则中明确，与解析策略（自顶向下或自底向上）一致。
 - 不会改变依赖图的合法性（不会引入新的循环依赖或破坏属性求值结果）。
- 若任一条件不满足，副作用就不受控，可能引发错乱（例如重复插入、提前生成错误的代码）



语法制导定义的基本概念



■ 为什么不能随意放置副作用？

$D \rightarrow T \text{ id}$

// 意图：把 id 加入符号表，类型就是 T.type

动作： `addType(id.name, T.type)`

- 合法情形（受控）：`addType` 在 T.type 已计算完之后、并且在无法回溯或取消的上下文中执行
- 非法情形（不受控）：在存在回溯的解析器里（如带回溯的递归下降/PEG 回溯）直接执行 `addType`，若解析失败回溯，会导致符号表状态与解析状态不同步 → 需要回滚或缓冲



语法制导定义的基本概念



- 受控副作用的例子
- $L \rightarrow E \ n \quad \text{print}(E.\text{val})$
 - 通过副作用打印出E的值
 - 总是在最后执行，而且不会影响其它属性的求值
- 变量声明的SDD中的副作用
 - addType将标识符的类型信息加入到标识符表中
 - 只要标识符不被重复声明，标识符的类型信息总是正确的

产生式	语义规则
1) $D \rightarrow T L$	$L.inh = T.type$
2) $T \rightarrow \mathbf{int}$	$T.type = \text{integer}$
3) $T \rightarrow \mathbf{float}$	$T.type = \text{float}$
4) $L \rightarrow L_1, \mathbf{id}$	$L_1.inh = L.inh$ $addType(\mathbf{id}.entry, L.inh)$
5) $L \rightarrow \mathbf{id}$	$addType(\mathbf{id}.entry, L.inh)$



抽象语法树与类型



- 抽象语法树的构造
- 基本类型和数组类型的L属性定义



抽象语法树与类型



抽象语法树（ Abstract Syntax Tree ）

VS.

具象语法树（ Concrete Syntax Tree ）



抽象语法树与类型



- 具象语法树（**Concrete Syntax Tree**）：按照文法规则推导得到的完整树，精确反映文法的层次结构
 - 特点：包含所有非终结符与终结符；结构冗余，贴近文法；强调推导过程
- 抽象语法树（**Abstract Syntax Tree**）：去除了文法中冗余的语法细节，只保留与程序语义相关的核心结构
 - 特点：节点表示语言的核心成分（如“加法”、“变量”）；忽略不必要的非终结符（如括号层次、文法辅助符号）；更接近于编译器内部的中间表示。



抽象语法树与类型



特性	具体语法树（ CST ）	抽象语法树（ AST ）
目的	展示文法推导过程	表达程序语义的核心结构
节点内容	所有非终结符和终结符	语义相关的构造，如运算符、标识符
冗余性	高，结构庞大	精简，去掉无用信息
应用场景	理论教学，文法验证	编译器前端，中间代码生成

CST = 全部语法细节（就像考试解题步骤）

AST = 最终简化结果（就像答案公式）

在编译器中，**AST** 是后续 语义分析、优化和代码生成的核心输入



抽象语法树与类型



- 语法树（**CST**）：层次复杂，包含大量冗余符号。
- 抽象语法树（**AST**）：精简，直接反映程序的语义结构。
- 在 **AST** 上进行属性计算，效率更高，逻辑更清晰。
- 应用场景：
 - 类型检查（type checking）
 - 作用域/符号表管理
 - 常量折叠、优化
 - 中间代码生成



抽象语法树与类型



■ 抽象语法树

- 每个结点代表一个语法结构；对应于一个运算符
- 结点的每个子结点代表其子结构；对应于运算分量
- 表示这些子结构按照特定方式组成了较大的结构
- 可以忽略掉一些标点符号等非本质的东西

■ 构造抽象语法树的SDD

■ 语法树的表示方法

- 每个结点用一个对象表示
- 对象有多个域
 - 叶子结点中只存放词法值
 - 内部结点中存放了op值和参数（通常指向其它结点）



抽象语法树与类型



- 在 **SDT** 中语义动作不仅用于生成 **AST**，还可：
 - 在遇到声明语句 时：向符号表插入条目
 - 在遇到变量使用 时：查询符号表进行检查
- 在 **AST** 中每个 标识符结点 都需要绑定符号表中的条目
- 符号表与 **AST** 协同工作：
 - **AST** 管理结构（树形层次）
 - 符号表管理名字（作用域与属性）



抽象语法树与类型



- 示例:

```
int x;  
int f(int y) {  
    return x + y;  
}
```

- AST 负责表达式树形结构

- 符号表记录: $x:\text{int}$ (全局), $y:\text{int}$ (局部), $f:\text{function}$

- 符号表是语法树之外的“语义支撑系统”



抽象语法树与类型



■ 符号表的基本概念

■ 定义

- 符号表是编译器中用于记录源程序中各种符号（变量、函数、类型等）的数据结构。
- 本质上是一个 映射表（名称 $\rightarrow$ 属性）

■ 作用

- 在语义分析阶段，保存标识符的语义信息
- 在后续的 中间代码生成、优化、目标代码生成 阶段提供支持。



抽象语法树与类型



- 符号表的基本概念
- 记录的信息
 - 符号名称 (Identifier)
 - 类型信息 (Type)
 - 作用域 (Scope)
 - 存储分配 (Memory Location)
 - 其他属性 (如函数参数列表、是否为常量等)



抽象语法树与类型



- 符号表的基本概念
- 符号表的用途
 - 检查语义合法性
 - 变量是否声明后使用？
 - 类型是否匹配？
 - 作用域管理
 - 支持嵌套作用域（如函数、代码块）
 - 支持代码生成
 - 为每个变量分配存储空间或寄存器
 - 优化辅助
 - 确定变量的生命周期，进行寄存器分配或常量传播



抽象语法树与类型



- 符号表的基本概念
- 常见实现方式
 - 哈希表（Hash Table）：查找效率高（ $O(1)$ ）
 - 链表（Linked List）：适合作作用域嵌套结构
 - 树（如平衡树）：适合需要有序遍历的情况
- 支持作用域的设计
 - 单表 + 作用域链：符号表栈（stack of tables）
 - 进入新作用域 → 新建符号表；退出作用域 → 弹出符号表



四、简答题（共 1 题，共 15 分）

给定一个表示四进制小数的文法 G ，其开始符号为 S ，并且包含以下四个产生式。小数部分可以为空，即 123 是合法的四进制小数。产生式如下：

得分	
评分人	

$$\begin{aligned} G(S): \quad S &\rightarrow Q_1 \mid Q_1.Q_2 \\ Q_1 &\rightarrow Q_1\text{Num} \mid \text{Num} \\ Q_2 &\rightarrow \text{Num}Q_2 \mid \text{Num} \\ \text{Num} &\rightarrow 0 \mid 1 \mid 2 \mid 3 \end{aligned}$$

- 1.（5 分）文法 G 中包含左递归，请消除左递归；
- 2.（5 分）请设计一个语法制导翻译方案来统计四进制小数中奇数数字（1 和 3）的个数。
例如 2.123 中奇数数字个数为 2；
- 3.（5 分）请设计一个语法制导翻译方案将四进制小数转换为二进制字符串。例如 2.123 转换为二进制字符串为”10.011011”。



四、简答题（共 1 题，共 15 分）

给定一个表示四进制小数的文法 G ，其开始符号为 S ，并且包含以下四个产生式。小数部分可以为空，即 123 是合法的四进制小数。产生式如下：

$G(S): \quad S \rightarrow Q_1 \mid Q_1.Q_2$
 $Q_1 \rightarrow Q_1Num \mid Num$
 $Q_2 \rightarrow NumQ_2 \mid Num$
 $Num \rightarrow 0 \mid 1 \mid 2 \mid 3$

- 1. (5 分) 文法 G 中包含左递归，请消除左递归；
- 2. (5 分) 请设计一个语法制导翻译方案来统计四进制小数中奇数数字 (1 和 3) 的个数。
例如 2.123 中奇数数字个数为 2；
- 3. (5 分) 请设计一个语法制导翻译方案将四进制小数转换为二进制字符串。例如 2.123 转换的二进制字符串为“10.011011”。

得分	
评分人	

1. 消除文法 G 的左递归，就是消除 Q_1 的左递归：

$Q_1 \rightarrow NumQ'_1$
 $Q'_1 \rightarrow NumQ'_1 \mid \epsilon$

更新后的文法 $G(S)$ 如下：

$G(S): \quad S \rightarrow Q_1 \mid Q_1.Q_2$
 $Q_1 \rightarrow NumQ'_1$
 $Q'_1 \rightarrow NumQ'_1 \mid \epsilon$
 $Q_2 \rightarrow NumQ_2 \mid Num$
 $Num \rightarrow 0 \mid 1 \mid 2 \mid 3$



四、简答题（共 1 题，共 15 分）

给定一个表示四进制小数的文法 G ，其开始符号为 S ，并且包含以下四个产生式。小数部分可以为空，即 123 是合法的四进制小数。产生式如下：

$$\begin{aligned} G(S): \quad S &\rightarrow Q_1 \mid Q_1.Q_2 \\ Q_1 &\rightarrow Q_1\text{Num} \mid \text{Num} \\ Q_2 &\rightarrow \text{Num}Q_2 \mid \text{Num} \\ \text{Num} &\rightarrow 0 \mid 1 \mid 2 \mid 3 \end{aligned}$$

1. (5 分) 文法 G 中包含左递归，请消除左递归；
2. (5 分) 请设计一个语法制导翻译方案来统计四进制小数中奇数数字 (1 和 3) 的个数。
例如 2.123 中奇数数字个数为 2；
3. (5 分) 请设计一个语法制导翻译方案将四进制小数转换为二进制字符串。例如 2.123 转换的二进制字符串为“10.011011”。

得分	
评分人	

2. 统计四进制数字中的奇数数字的个数的语义规则如下：

$$\text{Num} \rightarrow 1 \mid 3: \text{Num.count} = 1$$

$$\text{Num} \rightarrow 0 \mid 2: \text{Num.count} = 0$$

$$Q_1 \rightarrow \text{Num}Q'_1: Q_1.\text{count} = \text{Num.count} + Q'_1.\text{count}$$

$$Q'_1 \rightarrow \text{Num}Q'_1: Q'_1.\text{count} = \text{Num.count} + Q'_1.\text{count}$$

$$Q'_1 \rightarrow \epsilon: Q'_1.\text{count} = 0$$

$$Q_2 \rightarrow \text{Num}Q_2: Q_2.\text{count} = \text{Num.count} + Q_2.\text{count}$$

$$Q_2 \rightarrow \text{Num}: Q_2.\text{count} = \text{Num.count}$$



四、简答题（共 1 题，共 15 分）

给定一个表示四进制小数的文法 G ，其开始符号为 S ，并且包含以下四个产生式。小数部分可以为空，即 123 是合法的四进制小数。产生式如下：

$$\begin{aligned} G(S): \quad S &\rightarrow Q_1 \mid Q_1.Q_2 \\ Q_1 &\rightarrow Q_1\text{Num} \mid \text{Num} \\ Q_2 &\rightarrow \text{Num}Q_2 \mid \text{Num} \\ \text{Num} &\rightarrow 0 \mid 1 \mid 2 \mid 3 \end{aligned}$$

1. (5 分) 文法 G 中包含左递归，请消除左递归；
2. (5 分) 请设计一个语法制导翻译方案来统计四进制小数中奇数数字 (1 和 3) 的个数。
例如 2.123 中奇数数字个数为 2；
3. (5 分) 请设计一个语法制导翻译方案将四进制小数转换为二进制字符串。例如 2.123 转换的二进制字符串为“10.011011”。

得分	
评分人	

3. 将四进制小数转换为二进制字符串形式的语义规则如下：

$$\text{Num} \rightarrow 0 : \text{Num.bin} = \text{“00”}$$

$$\text{Num} \rightarrow 1 : \text{Num.bin} = \text{“01”}$$

$$\text{Num} \rightarrow 2 : \text{Num.bin} = \text{“10”}$$

$$\text{Num} \rightarrow 3 : \text{Num.bin} = \text{“11”}$$

$$Q_1 \rightarrow \text{Num}Q'_1 : Q_1.\text{bin} = \text{Num.bin} + Q'_1.\text{bin}$$

$$Q'_1 \rightarrow \text{Num}Q'_1 : Q'_1.\text{bin} = \text{Num.bin} + Q'_1.\text{bin}$$

$$Q'_1 \rightarrow \epsilon : Q'_1.\text{bin} = \text{“”}$$

$$Q_2 \rightarrow \text{Num}Q_2 : Q_2.\text{bin} = \text{Num.bin} + Q_2.\text{bin}$$

$$Q_2 \rightarrow \text{Num} : Q_2.\text{bin} = \text{Num.bin}$$



题目 5 (语法制导定义与翻译 (10=5+5 分))

已知程序的文法 $G[P]$ 如下:

$$P \rightarrow D$$
$$D \rightarrow D ; D \mid id : T \mid \text{proc } id ; D ; S$$

- (1) 请写一个语法制导定义, 输出程序中一共声明了多少个 id 。
- (2) 请写一个翻译模式, 输出程序中每个变量 id 的嵌套深度。



题目 5 (语法制导定义与翻译 (10=5+5 分))

已知程序的文法 $G[P]$ 如下:

$P \rightarrow D$

$D \rightarrow D ; D \mid id : T \mid \text{proc } id ; D ; S$

(1) 请写一个语法制导定义, 输出程序中一共声明了多少

(2) 请写一个翻译模式, 输出程序中每个变量 id 的嵌套范

解答:

(1)

产生式	语义规则
$P \rightarrow D$	$\text{Print}(D.c)$
$D \rightarrow D1 ; D2$	$D.c = D1.c + D2.c$
$D \rightarrow id : T$	$D.c = 1$
$D \rightarrow \text{proc } id ; D1 ; S$	$D.c = D1.c + 1$

(2)

$P \rightarrow \{D.d = 1\} D$

$D \rightarrow \{D1.d = D.d\} D1 ; \{D2.d = D.d\} D2$

$D \rightarrow id : T \{ \text{print}(D.d) \}$

$D \rightarrow \text{proc } id ; \{ D1.d = D.d + 1 \} D1 ; S$